Algorithms and Software Stack for Digital Twins

*Research date: 2026-10-18. **[?]** = could not confirm.*

1. Localization and mapping

LiDAR / LiDAR-inertial (LIO)

System	Strengths	Notes
FAST-LIO2	Fast, robust, the standard LIO baseline (Livox and spinning LiDARs)	Odometry only; add your own loop closure
Point-LIO	Survives aggressive motion	For handheld and legged platforms
FAST-LIVO2	Adds a camera; coloured maps; robust in corridors	
LIO-SAM	GPS and loop-closure factors built in	Older; needs careful tuning
KISS-ICP	Minimal tuning, no IMU needed	
GLIM	GPU, globally consistent, map-correction tool, GNSS extension, ROS 2	Strong pick for offline mapping

Visual / VIO

ORB-SLAM3 (GPLv3, multi-map), OpenVINS, VINS-Fusion. Newer learned options: DPVO, MASt3R-SLAM, VGGT-SLAM. These are promising but less proven at site scale.

Multi-session merging and place recognition

- Place recognition: Scan Context++, STD. For images: hloc, AnyLoc/MegaLoc **[?]**.
- Long-term maps: LT-mapper, maplab 2.0, Swarm-SLAM.
- Back ends: GTSAM, Ceres, g2o, with robust kernels to reject bad loops.

Georeferencing

Fuse RTK as factors in the pose graph. Indoors, use total-station control points. Handle coordinate systems with PROJ, and link to BIM through IFC 4.3 `IfcMapConversion`. Survey 3-5 checkpoints per capture and report RMSE.

2. 3D reconstruction

- **Photogrammetry:** COLMAP + GLOMAP (BSD), OpenMVS (AGPL), **WebODM** (best open-source drone pipeline), Meshroom. Commercial: RealityScan 2.x (free under $1M revenue), Pix4D, Metashape.
- **Feed-forward models:** DUSt3R/MASt3R, **VGGT** (CVPR 2025 best paper), **MapAnything** (metric scale), Depth Anything 3. Useful for initialisation and sparse captures. For survey accuracy, still refine with bundle adjustment and anchor to GCPs and LiDAR.

- **Gaussian splatting: gsplat** (Apache, recommended; the Inria original is non-commercial).
  - Geometry-accurate: 2DGS, PGSR, GOF.
  - Large scenes: Hierarchical-3DGS, CityGaussian v2, CityGS-X, BlitzGS (2026).
  - Seed with LiDAR points and add a depth loss to fix scale on textureless concrete.
  - SLAM-GS (SplaTAM, MonoGS) is still fragile at site scale. Prefer LIO poses, then offline GS.
- **Dynamic objects in GS:** SAM masks and robust losses (SpotLessSplats, WildGaussians).
- **Meshing:** TSDF (Open3D, VDBFusion), screened Poisson, GOF/2DGS. For BIM-like output, use RANSAC plane and primitive fitting.

3. Change detection and the 4D twin

1. **Scan-to-scan:**
   - Register each capture to a common control frame: TEASER++ / FPFH for coarse alignment, then small_gicp for fine.
   - Compare with C2C for quick checks and **M3C2** (py4dgeo, CloudCompare) for significance-aware distances.
   - Use voxel occupancy differencing to find added and removed material.
   - Earthworks: difference the DSMs (PDAL + GDAL) to get cut and fill volumes.
2. **Scan-to-BIM:**
   - Convert IFC to a mesh with IfcOpenShell, sample it to points, then register.
   - Matching structural primitives works well: KPConv column segmentation plus RANSAC plus ICP reaches ~2 cm RMSE (ISPRS 2025).
   - Compute signed cloud-to-mesh deviation per IFC GlobalId and colour it against tolerance bands (ACI 117 / DIN 18202).
3. **Progress:**
   - For each element, measure the share of its surface supported by scan points.
   - Use ray casting so occluded elements count as *unknown*, not *absent*.
   - Image-based: reproject BIM elements into photos and classify their state (formwork → rebar → poured).
   - Map elements to P6/MSP activities, then roll up to earned value.
   - There is no large public as-planned vs as-built benchmark, so expect to label your own data.
4. **Dynamic removal:** ERASOR, Removert, DUFOMap, BeautyMap; MapMOS/4DMOS online. Keep the removed dynamics as a separate tracked layer, because they are the safety data.

4. Semantic understanding

- **Point-cloud models:** Point Transformer V3, Sonata, Concerto (via Pointcept).
- **Open-vocabulary 3D:** OpenScene, OpenMask3D, ConceptGraphs, LangSplat.
- **2D:** YOLO11 (AGPL), RT-DETR/RF-DETR (Apache), Grounding DINO, YOLO-World, **SAM 2 / SAM 3** (concept-prompted segmentation, Nov 2025). Lift masks to 3D by projecting them through calibrated poses and voting across views.
- **Scene graphs:** Hydra, ConceptGraphs, HOV-SG. Align graph nodes with IFC spatial structure (Site, Building, Storey, Space).
- **Datasets:** SODA (20k images, 15 classes incl. PPE), MOCS (42k images, 174 sites), ACID (10k machine images), CIS, AIDCON (UAV), VCVW-3D (synthetic), PPE sets (SHWD, CHV).
- **Safety:**
  - Detection plus tracking (ByteTrack, BoT-SORT), ReID, and pose estimation (RTMPose).

    - Geofences around swing radii, PPE checks, equipment activity recognition.
    - Run on the edge with DeepStream/TensorRT, and blur faces before upload.

5. Data standards

Domain	Standard
BIM	IFC 4.3 (ISO 16739-1:2024); IFC5/IFCX still alpha
Raw scans	E57
Point clouds	LAS 1.4 / LAZ; COPC for cloud streaming
Geospatial streaming	OGC 3D Tiles 1.1
Assets and splats	glTF 2.0 + KHR_gaussian_splatting + SPZ (name being finalised **[?]**); PLY; SPZ
Scene composition	OpenUSD (Core Spec 1.0, Dec 2025)
City	CityGML 3.0 / CityJSON
Twin ontology	DTDL v3, Brick, RealEstateCore, NGSI-LD
Schedule	P6 XER/XML, MS Project XML, IfcWorkSchedule. Read with MPXJ
Sensors	ROS 2 MCAP, MQTT Sparkplug B, OPC UA

6. Platform

- **Geometry:** PDAL, Open3D, PCL, CloudCompare (CLI), py4dgeo, untwine, IfcOpenShell + Bonsai, CGAL, GDAL, laspy.
- **Visualization:**
    - CesiumJS: georeferenced tiles, plus splats with LOD (Apr 2026).
    - Potree: billion-point viewing.
    - three.js + 3d-tiles-renderer, with Spark or GaussianSplats3D for splats.
    - Also: deck.gl, Babylon, That Open Engine (IFC), SuperSplat, Unreal 5 + Cesium, Omniverse/Isaac Sim, iTwin.js.
- **IoT:** MQTT (EMQX/Mosquitto) → Kafka/Redpanda → TimescaleDB + PostGIS, Grafana, and Iceberg/Parquet on S3/MinIO.
- **Twin frameworks:**
    - Eclipse Ditto: good open-source core.
    - FIWARE Orion-LD.
    - Azure Digital Twins: low investment; Microsoft has moved to the Fabric digital twin builder (preview).
    - **AWS IoT TwinMaker: at risk, so avoid it.**
- **Robot:** ROS 2 Jazzy, MCAP, Foxglove, Rerun.

7. Reference pipeline

```
CAPTURE (edge)  →  INGEST  →  REGISTER/ALIGN  →  RECONSTRUCT  →
SEMANTICIZE  →  COMPARE/ANALYZE  →  SERVE  →  ALERT
```

```
LiDAR+IMU+cams     MCAP/E57    LIO poses,         COPC, gsplat
PTv3/Sonata,     M3C2, C2M per      CesiumJS,   rules on
(ROS 2), drone,    /LAZ/JPG    multi-session      (LiDAR-seeded),
SAM3+DINO lift,  IFC element,       Potree,     Ditto/Kafka:
TLS, 360, CCTV,    → S3/MinIO  merge (GLIM),      WebODM ortho,    dynamic
removal, progress → P6,     That Open,  tolerance,
IoT sensors;       MQTT→Kafka  RTK/GCP georef,    TSDF mesh,       scene
graph↔IFC  safety zones/PPE   Grafana,    schedule slip,
edge LIO, YOLO,                scan-to-BIM        → 3D Tiles/SPZ
UE5/Omni    geofence/PPE
blur, compress                 (TEASER+GICP)
```

Edge (Jetson)	Cloud / on-prem GPU
Drivers, PTP time sync, real-time LIO, PPE detection and tracking, geofence alerts (<1 s), blurring, compression, store-and-forward, local MQTT/Ditto cache	Global optimisation, merging, georeferencing, SfM, GS training, segmentation, scan-to-BIM, progress, 3D Tiles, databases, web

Cadence: safety runs in real time at the edge; progress scans daily or weekly; the visual twin refreshes weekly or at milestones.

8. Recommended open-source-first stack

Layer	Pick
Middleware	ROS 2 Jazzy + MCAP + Foxglove/Rerun
LIO	FAST-LIO2 (Point-LIO / FAST-LIVO2)
Global mapping	GLIM + glim_ext, or a custom GTSAM back end
Drone	WebODM
Feed-forward	MapAnything / VGGT
Visual twin	gsplat (LiDAR-seeded) + 2DGS/PGSR + Hierarchical-3DGS
Point clouds	PDAL + COPC, Open3D, CloudCompare, py4dgeo, small_gicp, TEASER++
BIM / schedule	IfcOpenShell, That Open Engine, MPXJ
Semantics	Pointcept (PTv3/Sonata/Concerto), SAM 3 + Grounding DINO / RF-DETR, ByteTrack
Dynamic removal	MapMOS / DUFOMap
Twin state	Eclipse Ditto
Telemetry	MQTT → Kafka/Redpanda → TimescaleDB + PostGIS, Grafana
Storage	MinIO/S3 + Iceberg; STAC-style catalog of capture epochs
Viewer	CesiumJS + Spark (splats) + Potree
Simulation	Unreal 5 + Cesium; Omniverse/Isaac Sim

Licensing watch-outs: ORB-SLAM3 is GPLv3. OpenMVS and Ultralytics are AGPL. Inria 3DGS is non-commercial (use gsplat). Check the SAM License terms. FAST-LIO2 is GPL-2.0 **[?]**.

9. Open uncertainties

1. AWS TwinMaker status.
2. Final glTF splat extension name.
3. Whether COPC is an OGC standard.
4. Lack of public progress benchmarks.
5. Feed-forward and SLAM-GS methods are not yet survey-grade.